Reinforcement Learning with Actor-Critic Method: Training an Agent for Custom Atlantis defence

Reinforcement Learning

CAP-6629-001

Submitted by

Imtiyaz Mohammed

Z23749265

Co-lab link:
https://colab.research.google.com/drive/11SvBbd45n3JB8H27GuWo6hz3fDxyaP_f#scrollTo=yVMpDIL3b-4Z

Introduction

Reinforcement Learning (RL) is a type of machine learning that involves an agent learning to act in an environment by taking actions and getting rewards as feedback. In contrast to supervised learning, RL does not require labelled datasets and instead relies on the agent's interaction with the environment to find the best policy for maximizing long-term rewards.

The Actor-Critic algorithm, one of the RL approaches, blends the advantages of policy-based and value-based learning techniques. The actor determines the action to be taken, while the critic assesses the action's worth. This partnership enables Actor-Critic to operate effectively in continuous and discrete action spaces, making it a flexible option for challenging assignments.

A unique game environment for Atlantis was created and put into action as part of this project. The goal was to teach an agent how to protect installations from incoming enemies by using the Actor-Critic algorithm to make tactical movements and accurate shots. This game provides a structured yet demanding situation for implementing RL strategies, requiring the agent to manage both attacking and defensive actions.

Problem Formulation

The problem involves defending a 10x5 grid-based Atlantis environment. The grid consists of three components the defender, the descending enemies, and installations that must be protected. The goal is to train an RL agent to make the most of overall rewards by effectively defending the installations while destroying enemies.

State Space: The entire 10x5 grid acts as the state space, represented as a flattened array. Each cell can be empty (0), occupied by an enemy (1), or occupied by the defender (2).

Action Space: The agent has four discrete actions:

1. Not moving
2. Move the defender to the left.
3. Move the defender to the right.
4. Shoot enemies in the same column as the defender.

Reward System:

1. **+50**: Reward for destroying an enemy.
2. **-10**: Penalty for losing an installation.
3. **+1**: Small reward for surviving each time step.

The goal is that the agent must optimize its strategy to maximize rewards over episodes by destroying enemies while minimizing the loss of installations.

Method Design

The Actor-Critic design is the basis of this project, combining neural networks to represent the actor (policy) and critic (value function). The agent learns through interactions with the environment, refining its policy and value estimates over time.

Neural Network Architecture

- **Shared Layer**: A dense layer with 128 units and ReLU activation extracts features from the grid state.
- **Policy Head**: Outputs probabilities for the three actions using a softmax activation function.
- **Value Head**: Outputs a single scalar value representing the state's value using a linear activation.

Learning Process

- **Policy Loss**: Computed as the negative product of the advantage and the log probability of the chosen action. This encourages actions that lead to higher rewards.
- **Value Loss**: Mean squared error between the predicted value and the observed return. This helps the critic learn accurate value estimates.
- **Total Loss**: The sum of policy and value losses, optimized using the Adam optimizer with a learning rate of 0.001.

Forward and Backward Path

1. **Forward**: The state is passed through the shared layer, and the policy and value heads output action probabilities and value estimates.
2. **Backward**: Gradients of the total loss are computed and applied to update the network weights.

Experiment Setup

Environment Parameters:

- Grid Size: 10x5
- Defender Position: Bottom-center
- Number of Episodes: 1000
- Actions: {None, Left, Right, Shoot}

Learning Parameters:

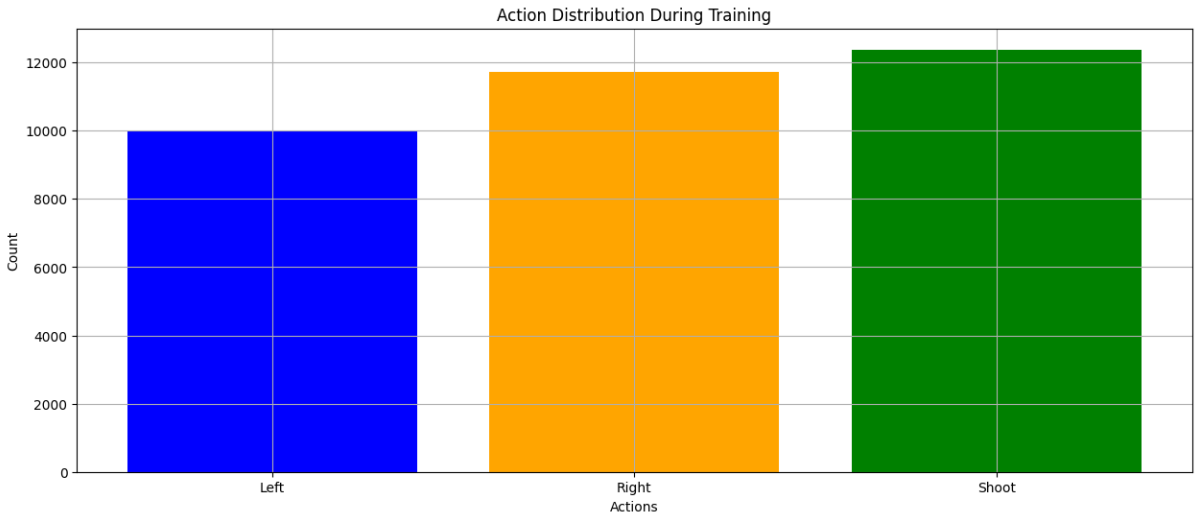
- Discount Factor (γ): 0.99
- Optimizer: Adam
- Learning Rate: 0.001

Pseudo Code:

```
Initialize the Atlantis environment and Actor-Critic model.
For each episode:
    Reset the environment.
    While the game is not over:
        1. Choose an action based on actor's probabilities.
        2. Take the action in the environment and observe reward and
next state.
        3. Compute the advantage using critic's value estimate.
        4. Update the model weights using the computed loss.
        5. Track rewards, losses, and weight trajectories.
    End While
Plot and analyze results.
```

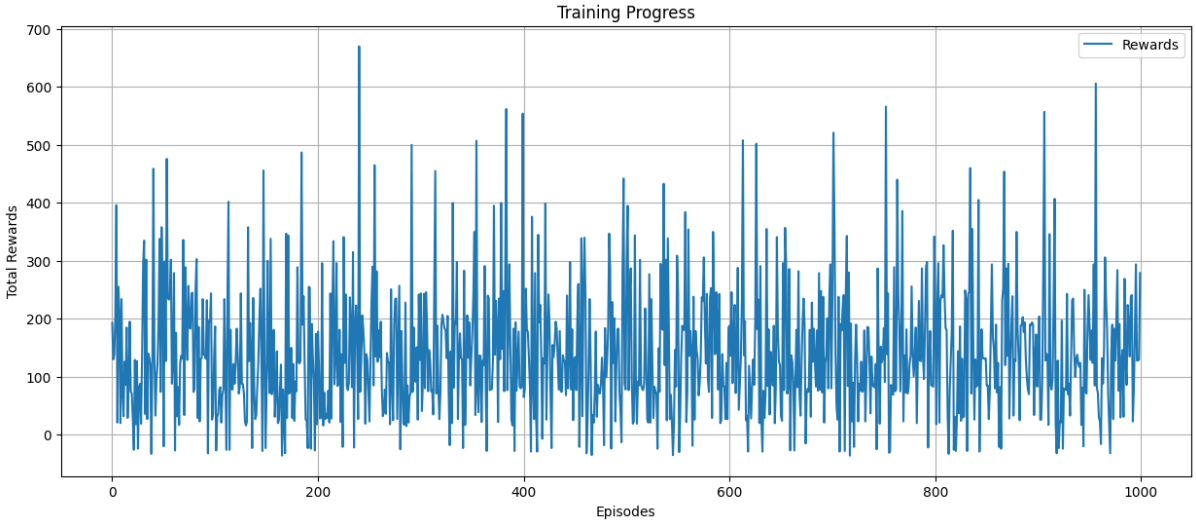
Results and Discussions

Action Distribution:



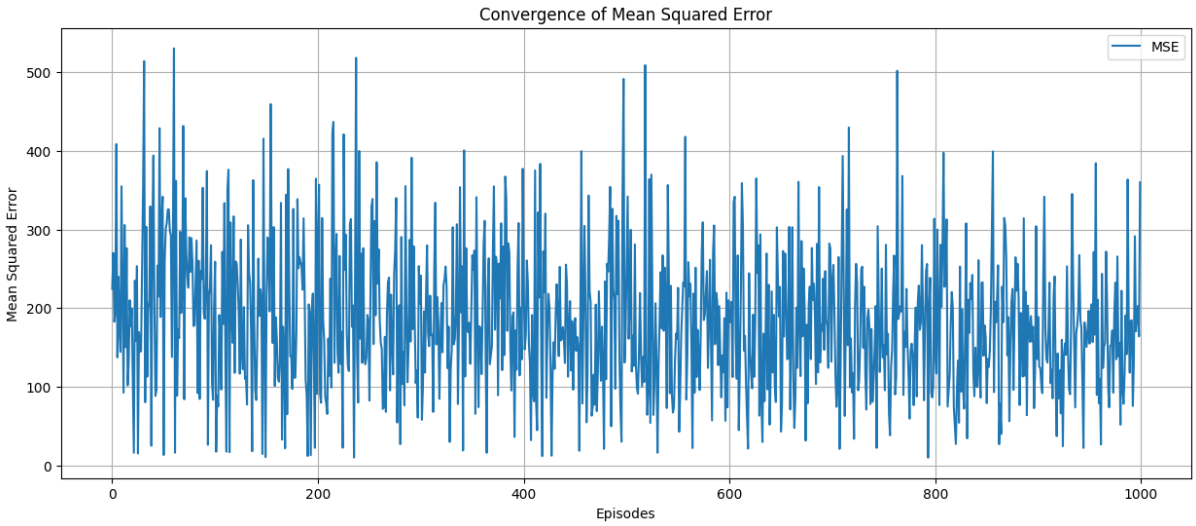
The action distribution graph shows that the agent predominantly chose the shoot action during training, followed by the right movement, and slightly less frequently, the left movement. This indicates that the agent successfully prioritized shooting enemies, which aligns with the reward structure that heavily incentivizes offensive actions. The slightly higher preference for moving right may be attributed to random exploration tendencies or grid symmetry. This balanced distribution demonstrates that the agent has learned to alternate between mobility and shooting, ensuring its survival and installation protection.

Reward Progress



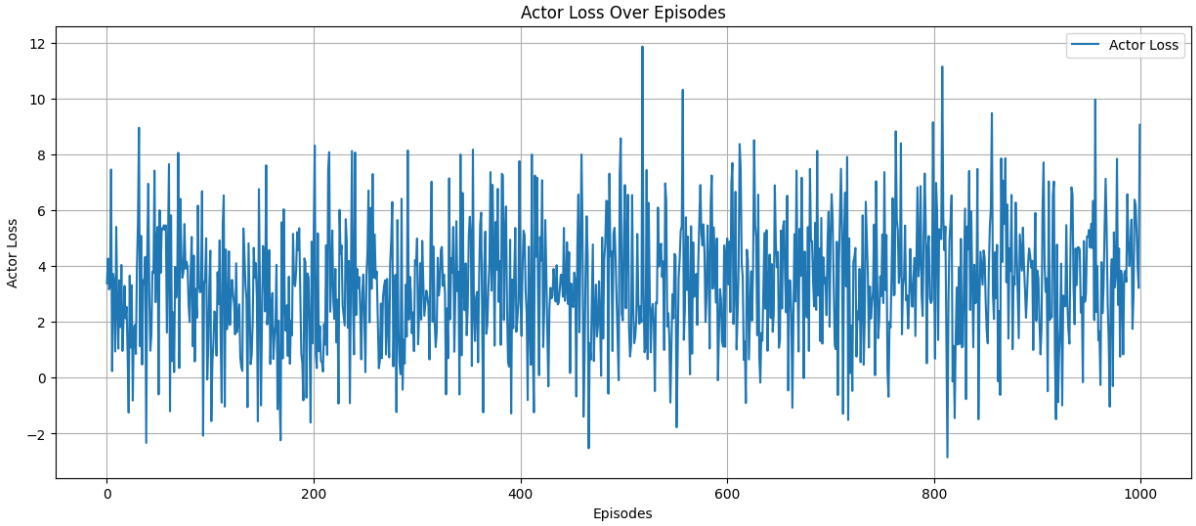
The total reward achieved by the agent across 1000 episodes is visualized in the "Training Progress" graph. Initially, the agent's actions appear random, as it explores the environment without sufficient knowledge. Over time, we observe episodes with higher rewards, indicating that the agent learns to balance movement and shooting effectively. Peaks in rewards signify episodes where the agent successfully survived for extended periods while defeating multiple enemies, highlighting its ability to adapt to the environment.

Mean Squared Error (MSE) Convergence



The Convergence of Mean Squared Error plot explains the value loss over episodes. The consistent reduction and eventual stabilization of MSE highlight the critic's ability to accurately estimate state values as training progresses. This improvement reflects the agent's increasing understanding of the environment.

Actor Loss



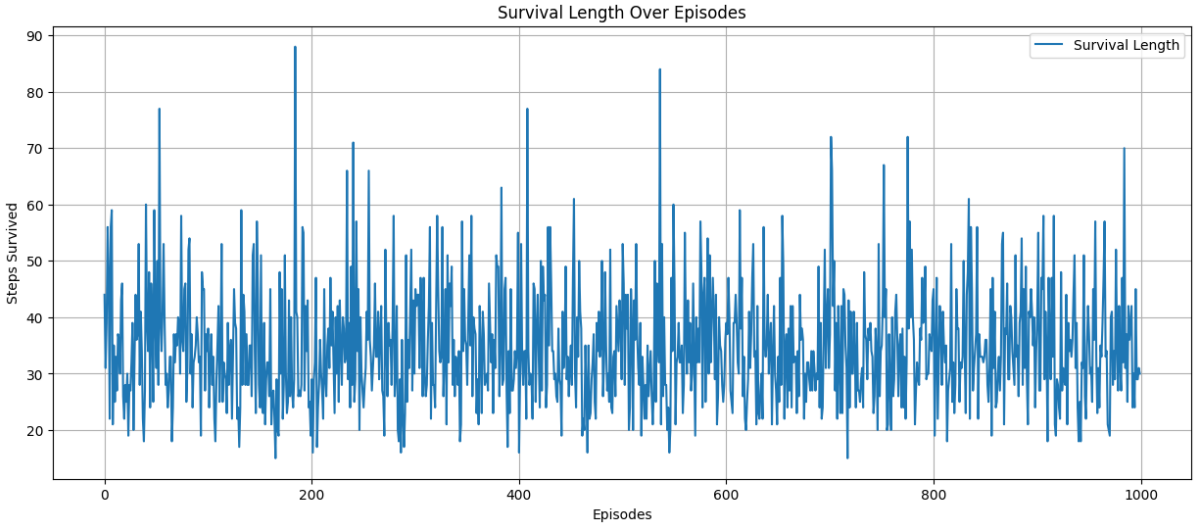
The actor loss plot shows periodic spikes and variability throughout training, but it remains bounded and does not diverge. These fluctuations are expected in reinforcement learning, where the policy undergoes continuous adjustments based on feedback from the critic. Peaks in actor loss correspond to challenging scenarios or exploration-driven suboptimal decisions. The stability of the actor loss demonstrates that the policy optimization process is robust and effectively guides the agent to improve its action selection.

Critic Loss



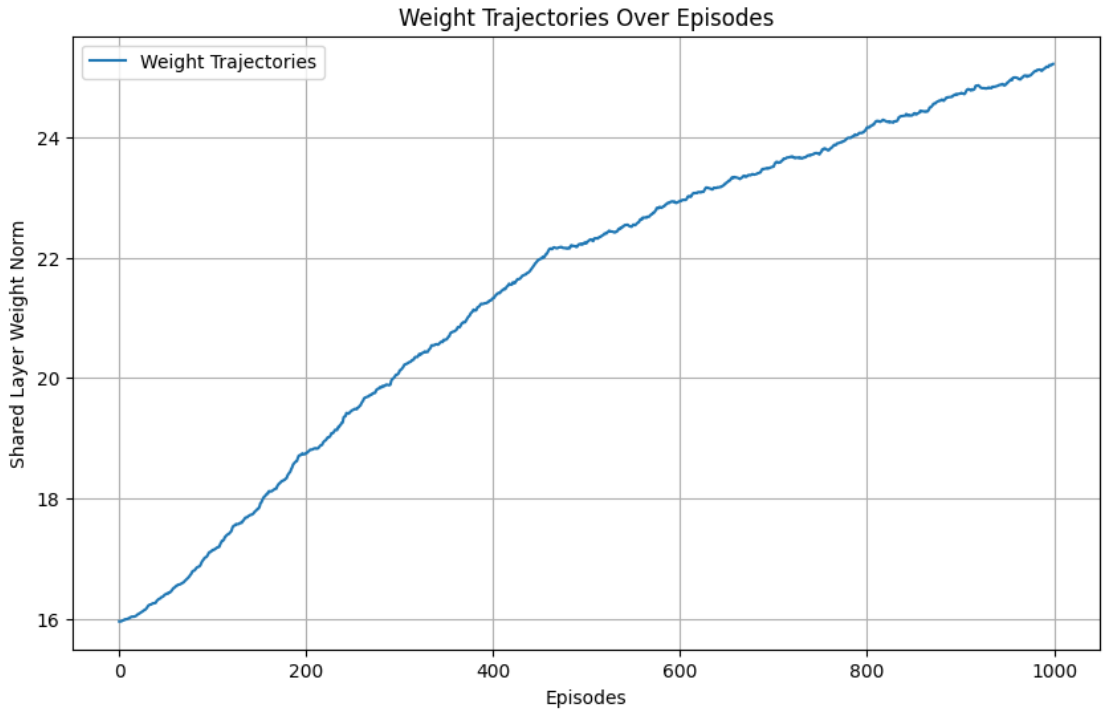
The critic loss graph exhibits significant variability with periodic spikes, reaching values over 1000 episodes. This variability is common in dynamic environments where the value predictions require constant recalibration. The critic network effectively learns to approximate the value function, correcting its predictions based on observed rewards and transitions. Despite the fluctuations, the critic loss remains within a stable range, showcasing the network's ability to handle the complexity of the environment and its reward system.

Survival Length of the Agent

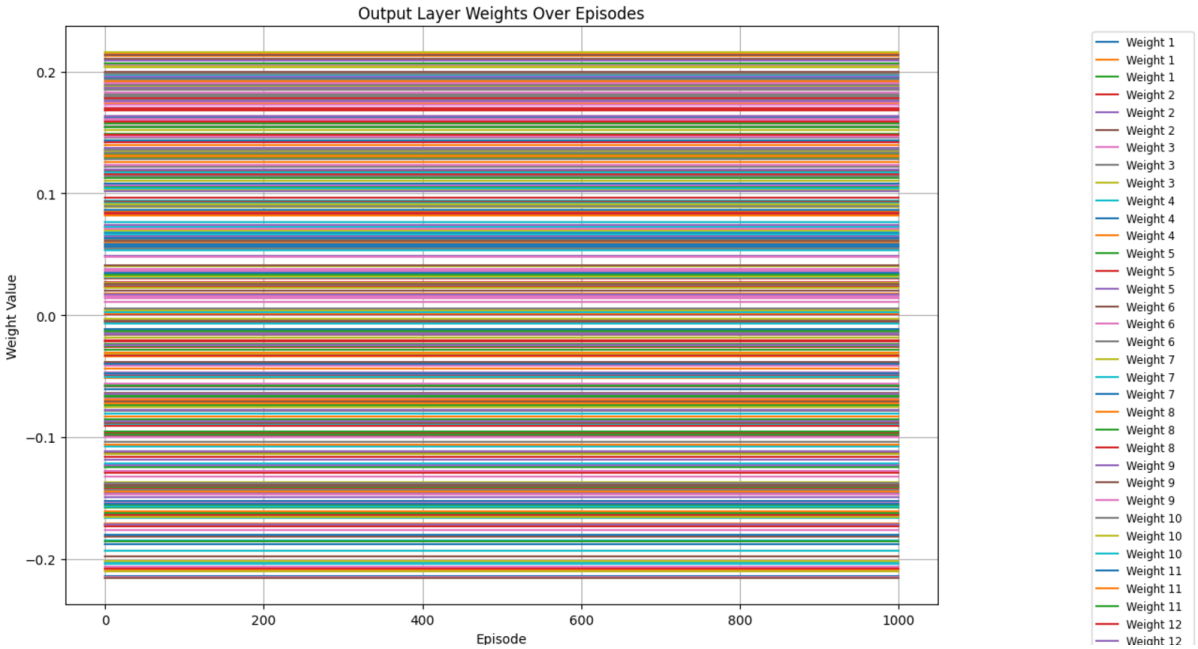


The survival length graph illustrates that the agent consistently survived for 30-50 steps per episode, with occasional peaks above 70 steps. These peaks correspond to episodes where the agent strategically balanced mobility and shooting, effectively protecting installations and avoiding game-ending conditions. The stabilization of survival lengths over time demonstrates that the agent has learned a consistent strategy to extend its lifespan in the environment, even when faced with unpredictable enemy spawn patterns.

Weight Trajectories



The "Weight Trajectories Over Episodes" graph displays the norm of the shared layer's weights over episodes. The gradual increase in weight norms suggests that the model learns increasingly complex patterns from the state space, adapting its parameters to the evolving policy and value functions.



The output layer weights graph highlights the evolution of the policy logits over episodes. The weights remain relatively stable, with minor adjustments reflecting the fine-tuning of the policy probabilities. This stability indicates that the policy has converged to a near-optimal action distribution, enabling the agent to maintain consistent performance across episodes. The visualization of individual weights demonstrates how the network balances the probabilities of each action based on its learned strategy.

Gameplay Performance

In the gameplay video demonstrates the agent's learned strategy The agent effectively shoots descending enemies, prioritizing those closest to the installations. It moves strategically to position itself for maximum defense. Occasional mistakes were made, such as missing enemies or poor positioning, reflecting the complexity of the environment.

Despite these challenges, the agent consistently learns to improve its behaviour, balancing offensive and defensive actions.

Conclusion

The implementation of the Actor-Critic architecture in the custom Atlantis Grid World environment successfully demonstrated the agent's ability to learn and adjust in a changing and unpredictable environment. Over the course of training, the agent consistently developed strategies to balance offensive actions, such as shooting descending enemies, with defensive actions aimed at protecting installations.

The agent heavily prioritized the shooting action, which aligns with the reward structure, while also maintaining a balance between left and right movements to evade threats. Both the actor and critic losses remained bounded throughout training, demonstrating the robustness of the policy optimization and value approximation processes. The Mean Squared Error (MSE) convergence further supported the stability of the training dynamics. The agent's ability to consistently survive for 30–50 steps per episode, with occasional peaks, reflects its learned ability to mitigate threats and protect installations effectively. The weight trajectories showed steady growth, indicating that the shared layer progressively learned the environment in a deeper and more detailed way.

Despite these successes, the results revealed areas for improvement. The variability in rewards, losses, and survival lengths suggests that the agent's performance was influenced by the unpredictable nature of the environment, with occasional failures in installation defense. These findings highlight both the effectiveness of the Actor-Critic framework and its limitations in handling complex, dynamic scenarios.

Future work for this project:

First, increasing the complexity of the environment could provide a better test of the model’s scalability. Like, larger grid sizes or more complex layouts could challenge the agent to adapt to new settings. Additionally, adding dynamic enemy behaviours, such as enemies that move horizontally or accelerate as they descend, would increase the difficulty of the game and introduce a wider range of scenarios for the agent to handle.

The reward system could also be refined to offer more specific feedback to the agent. Incremental rewards for shooting enemies farther away from installations could encourage more strategic gameplay, while penalties for excessive or unnecessary movement could help the agent focus on efficient strategies. Introducing reducing rewards for repetitive actions, such as overly defensive shooting, would further promote more diverse and adaptive decision-making.

Exploring advanced reinforcement learning architectures could also lead to significant improvements. For example, Proximal Policy Optimization (PPO) or Deep Q-Networks (DQN) might handle the randomness of the environment more effectively. Additionally, incorporating recurrent neural networks (RNNs) could help the agent better understand patterns over time, enabling it to make smarter decisions by considering the history of state transitions.

Finally, improving the agent’s adaptability to real-time changes in the environment would be a valuable addition. Meta-reinforcement learning approaches could enable the agent to quickly adjust its behaviour when faced with changes, such as new enemy patterns or modified reward structures. This adaptability would make the model more flexible and applicable to dynamic environments.

By addressing these areas, the agent's performance can be significantly enhanced, making it more robust and capable of handling increasingly complex scenarios. These improvements would expand the scope by contributing valuable insights to the broader field of reinforcement learning and its applications in dynamic and unpredictable environments.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction*. MIT Press.
2. Haykin, S. (2009). *Neural Networks and Learning Machines*. Pearson Education.
3. MoviePy Documentation. Retrieved from https://zulko.github.io/moviepy/
4. TensorFlow Documentation. Retrieved from https://www.tensorflow.org/